Complete two-group classification programs

These four unchanged files are required computational artifacts of the high-type and order-1024 applications. They are charged once within Topic A. Their entire byte content is printed below, including the 72 named marked presentations and all 107 source comparison rows. No old manuscript, saved census or private source file is read by the runner. A fresh output directory is mandatory. From this volume run

```
python3 -B -O required-code/phase78_n009_integration_run.py \
  --gap /path/to/gap --output-dir /fresh/output/path
```

The census independently generates all invariant kernels, checks every marked subgroup intersection and only then compares the resulting counts and named objects. The finite presentation checker obtains complete trivial-subgroup coset tables, verifies defining words and marked isomorphisms, and checks every finite universal row. The retained infinite decomposition is a separate written input. GAP and NQ's exact algorithms are the explicit computational trust boundary; this is not an end-to-end Lean certificate.

phase78_n009_integration_census.g

```
# Complete path-marked two-group census, parameterized by rank and
excess.
# Theorems Q5.5/Q5.6 prove candidate completeness and profile
recognition.
# Adapted from the reviewed phase21/22 ladder; original markings are
preserved.
# Every candidate, including profile rejections, receives all 4^d actual
meets.
# Four parameter cases share this program; no source count or name
filters it.
# The order-256 case additionally checks the four exact L035 seed word
patterns.
# Use a fresh working directory. Existing mathematical output paths are
refused.
if LoadPackage("nq")=fail then Error("NQ required");fi;
SizeScreen([240,40]);;
Run78:=function(d,excess78,prefix78)
 local F,f,rel,i,j,Q,epi,iso,G,r,lc,K,pc,key,levels,M,C,pi,V,
  maxes,L,N,known,nxt,k,edges,maxdim,dim,at,start,allChecks,ss,
  pars,good,accepted,hist,typ,kernelType,rev,selfdual,dualclasses,
  acceptedKernels,totalCandidates,gi,gj,pos,typcounts,typekey,
  inds,ambientRows,pcweights,ks,ps,kg,qp,kpars,kchecks,profileGood,
  rejectedProfiles,acceptedSelfTypes,selfTypeKey,facetOrders,facets,
  acceptedDetails,checkpointName,a78,b78,c78,r78,d078,z78,patterns78,seed
  78;
 if not (d>=3 and excess78>=1) then Error("Rank/excess input");fi;
 for k in [1..excess78] do
  if IsExistingFile(Concatenation(prefix78,".level",String(k),".json"))
  then Error("Existing checkpoint");fi;
 od;
 if IsExistingFile(Concatenation(prefix78,".accepted.json")) or
 IsExistingFile(Concatenation(prefix78,".seed.json")) then
 Error("Existing output");fi;
 seed78:=[];
 F:=FreeGroup(d);f:=GeneratorsOfGroup(F);rel:=List(f,x->x^2);
 for i in [1..d] do for j in [i+1..d] do
```

```
 if j-i>1 then Add(rel,Comm(f[i],f[j]));fi;
od;od;
Q:=F/rel;epi:=NqEpimorphismNilpotentQuotient(Q,excess78+2);
iso:=IsomorphismPcGroup(Image(epi));G:=Image(iso);
r:=List(GeneratorsOfGroup(Q),x->Image(iso,Image(epi,x)));
lc:=LowerCentralSeries(G);K:=lc[3];pc:=Pcgs(G);pcweights:=List([1..Lengt
h(pc)],t->2^(t-1));
if not ForAll(RelativeOrders(pc),n->n=2) then Error("Binary
canonical-pcgs key requires relative orders two");fi;
if Index(G,K)<>2^(2*d-1) then Error("Universal canonical quotient
order");fi;
key:=function(H)
 return
 List(CanonicalPcgs(InducedPcgsByGenerators(pc,GeneratorsOfGroup(H))),
  x->ExponentsOfPcElement(pc,x)*pcweights);
end;
Print("VERSION GAP=",GAPInfo.Version,"
NQ=",PackageInfo("nq")[1].Version,"\n");
Print("UNIVERSAL rank=",d," excess=",excess78," order=",Size(G),"
gamma3=",Size(K)," abelianization_type=",AbelianInvariants(K),"\n");
levels:=[K];start:=Runtime();
for k in [1..excess78] do
 known:=NewDictionary([],true);nxt:=[];edges:=0;maxdim:=0;at:=0;
 for M in levels do
  C:=ClosureGroup(CommutatorSubgroup(M,G),FrattiniSubgroup(M));
  pi:=NaturalHomomorphismByNormalSubgroup(M,C);V:=Image(pi);
  if not IsElementaryAbelian(V) then Error("Coinvariant square
  quotient");fi;
  dim:=LogInt(Size(V),2);maxdim:=Maximum(maxdim,dim);
  maxes:=MaximalSubgroups(V);
  if Length(maxes)<>2^dim-1 then Error("All hyperplanes");fi;
  for L in maxes do
   N:=PreImage(pi,L);edges:=edges+1;
   inds:=key(N);
   if LookupDictionary(known,inds)=fail then
    if Index(K,N)<>2^k or not IsNormal(G,N) then Error("Invariant ladder
    kernel");fi;
    AddDictionary(known,inds,Length(nxt)+1);Add(nxt,N);
   fi;
  od;
  at:=at+1;
  if at mod 100=0 then Print("PROGRESS depth=",k,"
  parents=",at,"/",Length(levels)," unique=",Length(nxt),"
  elapsed_ms=",Runtime()-start,"\n");fi;
 od;
 Print("LEVEL index=",2^k," parents=",Length(levels),"
 hyperplane_edges=",edges," unique=",Length(nxt),"
 max_coinvariant_dimension=",maxdim,"
 elapsed_ms=",Runtime()-start,"\n");
 levels:=nxt;
 if true then
  checkpointName:=Concatenation(prefix78,".level",String(k),".json");
  PrintTo(checkpointName,List(levels,key),"\n");
 fi;
od;
# Exact actual-IP checks for every final candidate, including every
meet.
ss:=Combinations([1..d]);allChecks:=0;accepted:=0;selfdual:=0;
```

```
ks:=List(ss,S->Intersection(K,Subgroup(G,r{S})));
kchecks:=0;rejectedProfiles:=0;acceptedSelfTypes:=[];acceptedDetails:=[]
;
acceptedKernels:=[];hist:=[];typcounts:=[];totalCandidates:=Length(level
s);
rev:=GroupHomomorphismByImages(G,G,r,Reversed(r));
if rev=fail or not IsBijective(rev) then Error("Marked reversal
automorphism");fi;
at:=0;
for N in levels do
 # The kernel quotient has order 2^excess78 and need not be abelian.
 qp:=NaturalHomomorphismByNormalSubgroup(K,N);kg:=Image(qp);
 if Size(kg)<>2^excess78 then Error("Small kernel quotient");fi;
 kpars:=List(ks,H->Image(qp,H));profileGood:=true;
 for i in [1..Length(ss)] do
  for j in [i+1..Length(ss)] do
   pos:=Position(ss,Intersection(ss[i],ss[j]));
   kchecks:=kchecks+1;
   if Intersection(kpars[i],kpars[j])<>kpars[pos] then
   profileGood:=false;break;fi;
  od;
  if not profileGood then break;fi;
 od;
 if not profileGood then
  rejectedProfiles:=rejectedProfiles+1;
  if at mod 100=0 then Print("PROFILE_PROGRESS
  candidates=",at,"/",totalCandidates," accepted=",accepted,"
  elapsed_ms=",Runtime()-start,"\n");fi;
 fi;
 pi:=NaturalHomomorphismByNormalSubgroup(G,N);Q:=Image(pi);
 f:=List(r,x->Image(pi,x));
 if Size(Q)<>2^(2*d-1+excess78) or Subgroup(Q,f)<>Q then
 Error("Candidate order/marking");fi;
 pars:=List(ss,S->Subgroup(Q,f{S}));good:=true;
 for i in [1..Length(ss)] do for j in [1..Length(ss)] do
  pos:=Position(ss,Intersection(ss[i],ss[j]));
  if Intersection(pars[i],pars[j])<>pars[pos] then good:=false;fi;
  allChecks:=allChecks+1;
 od;od;
 if good<>profileGood then Error("Kernel-profile and actual-IP criteria
 disagree");fi;
 if good then
  accepted:=accepted+1;Add(acceptedKernels,key(N));
  if Image(rev,N)=N then selfdual:=selfdual+1;fi;
  typ:=List([1..d-1],i->Order(f[i]*f[i+1]));
  if Minimum(typ)<4 then Error("Degenerate marking escaped canonical
  quotient");fi;
  kernelType:=AbelianInvariants(Image(pi,K)/DerivedSubgroup(Image(pi,K))
  );
  Add(hist,[Length(LowerCentralSeries(Q))-1,kernelType]);
  typekey:=Minimum(typ,Reversed(typ));Add(typcounts,typekey);
  if Image(rev,N)=N then Add(acceptedSelfTypes,typekey);fi;
  facetOrders:=[Size(Subgroup(Q,f{[1..d-1]})),Size(Subgroup(Q,f{[2..d]})
  )];
  Add(acceptedDetails,[key(N),typ,Length(LowerCentralSeries(Q))-1,kernel
  Type,Image(rev,N)=N,facetOrders]);
  if d=3 and excess78=3 and typ=[4,8] then
```

```
    a78:=f[1];b78:=f[2];c78:=f[3];r78:=b78*c78;d078:=(a78*b78)^2;z78:=r78
    ^4;
    patterns78:=[];
    if Comm(a78,r78^2)^2=One(Q) and Comm(a78,r78^4)=One(Q) then
    Add(patterns78,1);fi;
    if Comm(a78,r78^2)^2=z78 and Comm(a78,r78^4)=One(Q) then
    Add(patterns78,2);fi;
    if Comm(d078,r78^2)=z78 then Add(patterns78,3);fi;
    if Comm((r78^2)^a78,r78)=z78 then Add(patterns78,4);fi;
    if Length(patterns78)<>1 then Error("Unclassified or overlapping
    order-256 word pattern");fi;
    Add(seed78,[key(N),patterns78[1]]);
   fi;
  fi;
  at:=at+1;
  if at mod 100=0 then Print("IP_PROGRESS
  candidates=",at,"/",totalCandidates," accepted=",accepted,"
  elapsed_ms=",Runtime()-start,"\n");fi;
 od;
 dualclasses:=(accepted+selfdual)/2;
 Print("RESULT rank=",d," excess=",excess78,"
 candidates=",totalCandidates," ordered=",accepted,"
 selfdual=",selfdual," up_to_reversal=",dualclasses,"
 actual_meets=",allChecks," elapsed_ms=",Runtime()-start,"\n");
 Print("CLASS_KERNEL_ABELIANIZATION_HISTOGRAM ",Collected(hist),"\n");
 Print("ORDERED_CANONICAL_TYPE_HISTOGRAM ",Collected(typcounts),"\n");
 Print("SELFDUAL_TYPE_HISTOGRAM ",Collected(acceptedSelfTypes),"\n");
 Print("KERNEL_PROFILE_CHECKS comparisons=",kchecks,"
 rejected=",rejectedProfiles," directly_verified=",accepted,"\n");
 if true then
  PrintTo(Concatenation(prefix78,".accepted.json"),acceptedDetails,"\n");
 fi;
 PrintTo(Concatenation(prefix78,".seed.json"),seed78,"\n");
 if Length(seed78)>0 then Print("SEED_WORD_PATTERNS
 ",Collected(List(seed78,x->x[2])),"\n");fi;
 Print("PASS: complete invariant-kernel ladder and all-candidate
 actual-IP cross-check; counts are output, never candidate filters.\n");
 return rec(rank:=d,ordered:=accepted,selfdual:=selfdual,dualclasses:=dua
 lclasses,kernel_keys:=acceptedKernels);
end;;
if not IsBound(cases78) then cases78:=[[3,3],[3,5],[4,3],[5,1]];fi;
results78:=[];
for case78 in cases78 do
 Add(results78,Run78(case78[1],case78[2],Concatenation("phase78_d",String
 (case78[1]),"_k",String(case78[2]))));
od;
QUIT;
```

phase78_n009_integration_data.g

```
# Primary Atlas marked presentations; names are definitions, not a
completeness input.
# Each word is a string of generator indices 1,...,5, in displayed order.
# Exact source URL and captured-primary SHA256 precede every mathematical
record.
atlas78:=[
```

```
# https://www.abstract-polytopes.com/atlas/128/916/2.html
sha256=bbc4b3daca13f8719be87e21c622c7ac99dcc14af9044bd207705266d06a6d8c
rec(label:="{16,4}*128a",order:=128,typ:=[16, 4],words:=["11", "22",
"33", "1313", "12321232", "23232323",
"12121212121212121212121212121212"]),
# https://www.abstract-polytopes.com/atlas/128/922/2.html
sha256=14afde20067e7c9dc86bd4a772593bf38028834a97404a92f71bf2cbb83c4ced
rec(label:="{16,4}*128b",order:=128,typ:=[16, 4],words:=["11", "22",
"33", "1313", "23232323", "13232121323212", "212132121212121213"]),
# https://www.abstract-polytopes.com/atlas/256/5090/2.html
sha256=f2973f3db2e24e5666739bae1a3d2ce819fc6ec820b50ba042ed9d882a270558
rec(label:="{16,4}*256a",order:=256,typ:=[16, 4],words:=["11", "22",
"33", "1313", "23232323", "13232121323212",
"12121212121212121212121212121212"]),
# https://www.abstract-polytopes.com/atlas/256/5102/2.html
sha256=128af9dd98fb01072c66c15eac3626111f77a5e048205af30f4820690bbca7a7
rec(label:="{16,4}*256b",order:=256,typ:=[16, 4],words:=["11", "22",
"33", "1313", "23232323", "1212123212121232", "1232123212321232",
"21212312132132132121"]),
# https://www.abstract-polytopes.com/atlas/256/5298/2.html
sha256=12667f4e841c06bc1d5de167d44bbb1715341b0fd1e6d5f56b49546cf3210261
rec(label:="{16,8}*256a",order:=256,typ:=[16, 8],words:=["11", "22",
"33", "1313", "1231212312", "2323232323232323"]),
# https://www.abstract-polytopes.com/atlas/256/5302/2.html
sha256=e00e7a2a8ed96a988890fc6f29c9814f9a5349e7890d4d09e4304c236208b18d
rec(label:="{16,8}*256b",order:=256,typ:=[16, 8],words:=["11", "22",
"33", "1313", "12323121232312", "1212321232121212", "2323232323232323"]),
# https://www.abstract-polytopes.com/atlas/256/5304/3.html
sha256=4743105a3e8e20e4c2b955c9e7911a126c5373dfac19d2320cdec344e043a8b4
rec(label:="{16,8}*256c",order:=256,typ:=[16, 8],words:=["11", "22",
"33", "1313", "3123231232", "1212123212121232",
"12121212121212121212121212121212"]),
# https://www.abstract-polytopes.com/atlas/256/5304/4.html
sha256=5deb92ae5dbe5f671e64d42d32b92cfa265e743b216a0eb57e549915834dece9
rec(label:="{16,8}*256d",order:=256,typ:=[16, 8],words:=["11", "22",
"33", "1313", "12321232", "2323232323232323",
"12121212121212121212121212121212"]),
# https://www.abstract-polytopes.com/atlas/256/5312/3.html
sha256=6f43330ff2fff0e07165f2569c79d54863a6b240bd4d7015dfa2d749261cf1f9
rec(label:="{16,8}*256e",order:=256,typ:=[16, 8],words:=["11", "22",
"33", "1313", "12323121232312", "1212123212121232", "2323232323232323",
"12121321212132132132"]),
# https://www.abstract-polytopes.com/atlas/256/5312/4.html
sha256=102ea219f7655cc4799aa7cb9f9241d64ec47b43ed42cb11125072ebcff0c2fa
rec(label:="{16,8}*256f",order:=256,typ:=[16, 8],words:=["11", "22",
"33", "1313", "12323121232312", "1212123212121232", "2323232323232323",
"12123121212132121212"]),
# https://www.abstract-polytopes.com/atlas/256/6649/2.html
sha256=a5fc58ebc37b74bce00039923b101bdbe470f26d3bea3f2e1633caf25e5dd8c7
rec(label:="{32,4}*256a",order:=256,typ:=[32, 4],words:=["11", "22",
"33", "1313", "12321232", "23232323",
"12121212121212121212121212121212121212121212121212121212121212121212121212121212"]),
# https://www.abstract-polytopes.com/atlas/256/6655/2.html
sha256=ba344f7162834a36afb89c517e1095b4c1167155691f799663b105fc3d463df2
rec(label:="{32,4}*256b",order:=256,typ:=[32, 4],words:=["11", "22",
"33", "1313", "23232323", "13232121323212",
"12121212121212121213212132121212"]),
```

```
# https://www.abstract-polytopes.com/atlas/128/916/1.html
sha256=6575789b0dad2f7052958a92844a1e40811a8ea8c7e6063a537f400e797d396a
rec(label:="{4,16}*128a",order:=128,typ:=[4, 16],words:=["11", "22",
"33", "1313", "12121212", "12321232",
"232323232323232323232323232323"]),
# https://www.abstract-polytopes.com/atlas/128/922/1.html
sha256=31d8b42bd323b4508c6f01770cf3fef3135b245389b828b6e5a2af09856e41e6
rec(label:="{4,16}*128b",order:=128,typ:=[4, 16],words:=["11", "22",
"33", "1313", "12121212", "23123212132321", "2323232323123213232"]),
# https://www.abstract-polytopes.com/atlas/256/5090/1.html
sha256=37b1aba9f879c0870525f62d5c0b421125db0803208f2eba372fbaa446308070
rec(label:="{4,16}*256a",order:=256,typ:=[4, 16],words:=["11", "22",
"33", "1313", "12121212", "23123212132321",
"232323232323232323232323232323"]),
# https://www.abstract-polytopes.com/atlas/256/5102/1.html
sha256=b497484ccb716e3f5bba6d8c12d900f6c82f8d194ffb5f35c812d260a6c17976
rec(label:="{4,16}*256b",order:=256,typ:=[4, 16],words:=["11", "22",
"33", "1313", "12121212", "1232123212321232", "1232323212323232",
"323232323123123121"]),
# https://www.abstract-polytopes.com/atlas/256/6649/1.html
sha256=3ded1a80606ade68594e27a072f3126e0aaa7e144cfade829aea52ccdea5cdff
rec(label:="{4,32}*256a",order:=256,typ:=[4, 32],words:=["11", "22",
"33", "1313", "12121212", "12321232",
"232323232323232323232323232323232323232323232323232323232323232323"]),
# https://www.abstract-polytopes.com/atlas/256/6655/1.html
sha256=c9d7ce96ad5d81606c20e396fd33b97182cf567e9c54bd8ce573e7625573e4d0
rec(label:="{4,32}*256b",order:=256,typ:=[4, 32],words:=["11", "22",
"33", "1313", "12121212", "23123212132321",
"232323232323232323232323232312321323"]),
# https://www.abstract-polytopes.com/atlas/32/27/1.html
sha256=226ed85be2487dec1482158e430263c1703dc5f7d0cbf4f2b42c15555a419917
rec(label:="{4,4}*32",order:=32,typ:=[4, 4],words:=["11", "22", "33",
"1313", "12121212", "12321232", "23232323"]),
# https://www.abstract-polytopes.com/atlas/64/138/1.html
sha256=721fd35e9c6c2fe05e072e385d34b96d81b982dd24b682fafec424ba0abdb7a9
rec(label:="{4,4}*64",order:=64,typ:=[4, 4],words:=["11", "22", "33",
"1313", "12121212", "23232323", "312312312312"]),
# https://www.abstract-polytopes.com/atlas/128/928/1.html
sha256=5617f7b523b391005e4a66081507f2200c05ce7efb11c863c922b0ccd41b6124
rec(label:="{4,4}*128",order:=128,typ:=[4, 4],words:=["11", "22", "33",
"1313", "12121212", "23232323", "1232123212321232"]),
# https://www.abstract-polytopes.com/atlas/256/6661/1.html
sha256=f4b60a47adcd02659cb6c50d07b7e8b6f0aac5c89fad085705aa07494062bfa1
rec(label:="{4,4}*256",order:=256,typ:=[4, 4],words:=["11", "22", "33",
"1313", "12121212", "23232323", "3123123123123123123123"]),
# https://www.abstract-polytopes.com/atlas/64/128/1.html
sha256=77108260ce92b7a0404d306dc9364d333e2631dae3685d1a6c5019f5df39f0a4
rec(label:="{4,8}*64a",order:=64,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "12321232", "2323232323232323"]),
# https://www.abstract-polytopes.com/atlas/64/134/1.html
sha256=b94066b08eed5d66eb5ad7a45fb4172001c90020a35c5db0bed342a9a9c45540
rec(label:="{4,8}*64b",order:=64,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "3123231232"]),
# https://www.abstract-polytopes.com/atlas/128/327/1.html
sha256=970502a4caafbc0061c31ba75d59e3ac5b41b2cd368fd090557eb18bb06c8867
rec(label:="{4,8}*128a",order:=128,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "23123212132321", "2323232323232323"]),
```

```
# https://www.abstract-polytopes.com/atlas/128/928/2.html
sha256=ed51a0dd6a642ef5a81becd3a71b939014b736cd25a5fe225822906c1b8c7163
rec(label:="{4,8}*128b",order:=128,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "312312312312", "1232123212321232"]),
# https://www.abstract-polytopes.com/atlas/256/5078/1.html
sha256=0cb05fa3ed7cd8c41bf9b102a465a74e246022d6741148586789ed63d22d4a68
rec(label:="{4,8}*256a",order:=256,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "1232123212321232", "1232323212323232",
"2323232323232323"]),
# https://www.abstract-polytopes.com/atlas/256/6661/2.html
sha256=63ddd954ed300765714181e7f3647a389f5f2a79b6fea9693790533e07dbc6e9
rec(label:="{4,8}*256b",order:=256,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "312312312312", "2323232323232323"]),
# https://www.abstract-polytopes.com/atlas/256/6665/1.html
sha256=912e4da58975ece856295b1e2412e3e3cb0f1abddb3e0b2c0d84fb6793d5645d
rec(label:="{4,8}*256c",order:=256,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "1232323212323232", "2323232323232323",
"2123212323212321323"]),
# https://www.abstract-polytopes.com/atlas/256/6665/2.html
sha256=5303148e89fbfaacce9527c327be2b33abaf4583706c5880d62168da796a0706
rec(label:="{4,8}*256d",order:=256,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "2323232323232323", "2312123232132321"]),
# https://www.abstract-polytopes.com/atlas/256/5298/1.html
sha256=ef8854fa3725517ddfaa02d62d2736ffd67cd0ce490519ea4d3dbb2cade4e7cc
rec(label:="{8,16}*256a",order:=256,typ:=[8, 16],words:=["11", "22",
"33", "1313", "3123231232", "12121212121212"]),
# https://www.abstract-polytopes.com/atlas/256/5302/1.html
sha256=8661ee43431710cc6aefa692b3ec91a21be818a1615f0bcfd599532621af0273
rec(label:="{8,16}*256b",order:=256,typ:=[8, 16],words:=["11", "22",
"33", "1313", "12323121232312", "3232121232121232", "12121212121212"]),
# https://www.abstract-polytopes.com/atlas/256/5304/1.html
sha256=6edea98497d1c6494e32ab14d8ace26db2416282f64b268331649f29a8d25bb3
rec(label:="{8,16}*256c",order:=256,typ:=[8, 16],words:=["11", "22",
"33", "1313", "1231212312", "12121212121212",
"2323232323232323232323232323232323"]),
# https://www.abstract-polytopes.com/atlas/256/5304/2.html
sha256=430077132c84a2ccf8f84de03be5b3f29c0e5114715f3e7f231be8bbb1bf7c4c
rec(label:="{8,16}*256d",order:=256,typ:=[8, 16],words:=["11", "22",
"33", "1313", "12321232", "12121212121212",
"2323232323232323232323232323232323"]),
# https://www.abstract-polytopes.com/atlas/256/5312/1.html
sha256=13554272549558a51c9a354dc1795650cb1a408cf82fd2e17b050de7a95c7d9d
rec(label:="{8,16}*256e",order:=256,typ:=[8, 16],words:=["11", "22",
"33", "1313", "12323121232312", "12121212121212", "12121232121232",
"23231231231232323123"]),
# https://www.abstract-polytopes.com/atlas/256/5312/2.html
sha256=8a007116bdaaecf77719281c3e5851a9b84b673fa3cd5f268a6bfa04bcaf3852
rec(label:="{8,16}*256f",order:=256,typ:=[8, 16],words:=["11", "22",
"33", "1313", "12323121232312", "12121212121212", "12121232121232",
"3213232323232132"]),
# https://www.abstract-polytopes.com/atlas/64/128/2.html
sha256=509ded6ad933dd1a54a042f1c09e8f71f3dc5e5a9e402ee20b3b18ccc2883734
rec(label:="{8,4}*64a",order:=64,typ:=[8, 4],words:=["11", "22", "33",
"1313", "12321232", "23232323", "12121212121212"]),
# https://www.abstract-polytopes.com/atlas/64/134/2.html
sha256=854acdb58b686b8dde68a01a9d7cfa9ea1d65ab400dd88d9de3f4807ff831b7f
rec(label:="{8,4}*64b",order:=64,typ:=[8, 4],words:=["11", "22", "33",
"1313", "23232323", "1231212312"]),
```

```
# https://www.abstract-polytopes.com/atlas/128/327/2.html
sha256=d8dae6e472703dd52a96115f53adad40b3da0cd1632eb03ca0b046abc33f81d1
rec(label:="{8,4}*128a",order:=128,typ:=[8, 4],words:=["11", "22", "33",
"1313", "23232323", "13232121323212", "1212121212121212"]),
# https://www.abstract-polytopes.com/atlas/128/928/3.html
sha256=233f8cb33aee7120eda4d789888b904940d3e0299260c881cb78b3a1a88790ed
rec(label:="{8,4}*128b",order:=128,typ:=[8, 4],words:=["11", "22", "33",
"1313", "23232323", "312312312312", "1212121212121212",
"12121232121232"]),
# https://www.abstract-polytopes.com/atlas/256/5078/2.html
sha256=33ecd235667dcf98b5c56c35c2b4b2cd919026688d8d15944d92d3ec6ea1fb76
rec(label:="{8,4}*256a",order:=256,typ:=[8, 4],words:=["11", "22", "33",
"1313", "23232323", "12121212121212", "1212123212121232",
"12321232123212321232"]),
# https://www.abstract-polytopes.com/atlas/256/6661/3.html
sha256=592afee9ae5f06cc0c7af768bde8779f4f57c2d8ee43db9f187fdcd2d6ec6e06
rec(label:="{8,4}*256b",order:=256,typ:=[8, 4],words:=["11", "22", "33",
"1313", "23232323", "312312312312", "12121212121212"]),
# https://www.abstract-polytopes.com/atlas/256/6665/3.html
sha256=0ef16ce13d96eee9b01e8a528bf0e61b54ee738ce67d068c8e534ac5621a9250
rec(label:="{8,4}*256c",order:=256,typ:=[8, 4],words:=["11", "22", "33",
"1313", "23232323", "12121212121212", "1212123212121232",
"2121321232123212321213"]),
# https://www.abstract-polytopes.com/atlas/256/6665/4.html
sha256=03ec97313d17c0b0f812dac3670b7c301eb9860d03bfeb5702bdfb120ff5a404
rec(label:="{8,4}*256d",order:=256,typ:=[8, 4],words:=["11", "22", "33",
"1313", "23232323", "12121212121212", "1212323123212123232"]),
# https://www.abstract-polytopes.com/atlas/128/351/1.html
sha256=9093e3087b2c8ce6dc31c2c27f5f5fc1ca0b6cd89d6779384f0cd58b05d48d09
rec(label:="{8,8}*128a",order:=128,typ:=[8, 8],words:=["11", "22", "33",
"1313", "1231212312", "1212121212121212", "2323232323232323"]),
# https://www.abstract-polytopes.com/atlas/128/351/2.html
sha256=ce4c748a19b93e23922aefb13fc12d10fc09b54dc12cb699a4fd23ad7261f3bb
rec(label:="{8,8}*128b",order:=128,typ:=[8, 8],words:=["11", "22", "33",
"1313", "12321232", "12121212121212", "2323232323232323"]),
# https://www.abstract-polytopes.com/atlas/128/351/3.html
sha256=25f2d45c51102a966553d5e68c0d476cf1c15cc8854a800b23c204ed0e67bcf2
rec(label:="{8,8}*128c",order:=128,typ:=[8, 8],words:=["11", "22", "33",
"1313", "3123231232", "1212121212121212", "1212123212121232"]),
# https://www.abstract-polytopes.com/atlas/128/387/1.html
sha256=bfaa09d371f973f383ee656cba4b756aec032d8287df9d2054ad1edcfda13de9
rec(label:="{8,8}*128d",order:=128,typ:=[8, 8],words:=["11", "22", "33",
"1313", "12123232132132"]),
# https://www.abstract-polytopes.com/atlas/256/722/1.html
sha256=c2a84665c8de0bd59c1488cfd2e8d63aadcde7ed3f030535d1cdec8b282a9b4e
rec(label:="{8,8}*256a",order:=256,typ:=[8, 8],words:=["11", "22", "33",
"1313", "12323121232312", "12121212121212", "1212123212121232",
"2323232323232323"]),
# https://www.abstract-polytopes.com/atlas/256/5078/3.html
sha256=0ce9f52877b6f9aeb536ddbdaaf8fecacb889ba0feff8689dbf2a9dfeeb09b74
rec(label:="{8,8}*256b",order:=256,typ:=[8, 8],words:=["11", "22", "33",
"1313", "312312312312", "1212121212121212", "1212123212121232",
"12321232123212321232"]),
# https://www.abstract-polytopes.com/atlas/256/5084/1.html
sha256=d522856bb0abd2b66899900197829fe0c2f0ee73c63acd0c1184857f69cedc0b6
rec(label:="{8,8}*256c",order:=256,typ:=[8, 8],words:=["11", "22", "33",
"1313", "12323212132132", "1212121212121212", "1212123212121232",
"2323232323232323"]),
```

```
# https://www.abstract-polytopes.com/atlas/256/5084/2.html
sha256=0d6eb5f80fcf1fe0e5d5c9b74138941ab9515338e39525be01909daf25f90749
rec(label:="{8,8}*256d",order:=256,typ:=[8, 8],words:=["11", "22", "33",
"1313", "12323123123212", "1212121212121212", "1232123212321232"]),
# https://www.abstract-polytopes.com/atlas/256/6665/5.html
sha256=d806f95edac2ad28c0c4f3eba6a85e212c40983a119c1a742a4f2eed30510106
rec(label:="{8,8}*256e",order:=256,typ:=[8, 8],words:=["11", "22", "33",
"1313", "312312312312", "1212121212121212", "1212123212121232",
"2323232323232323", "212132123212321213"]),
# https://www.abstract-polytopes.com/atlas/256/6665/6.html
sha256=9608cf64536502e461675a5d051d9b051bbeda388b1add28e7f2e86ee224e1e4
rec(label:="{8,8}*256f",order:=256,typ:=[8, 8],words:=["11", "22", "33",
"1313", "312312312312", "1212121212121212", "1212323212123232",
"1232323212323232", "121321212321232312"]),
# https://www.abstract-polytopes.com/atlas/256/6669/1.html
sha256=fb23aa0d50ac8258fe9b00b0c06d34dd89f69b78024ffe7361be86b6225dde36
rec(label:="{8,8}*256g",order:=256,typ:=[8, 8],words:=["11", "22", "33",
"1313", "12323123123212", "1212121212121212", "2323232323232323",
"2312323212132123232"]),
# https://www.abstract-polytopes.com/atlas/256/6669/2.html
sha256=ac242b8b3d59337a8720f9016349f204865ba4de430b946ce37868a6cf931c0c
rec(label:="{8,8}*256h",order:=256,typ:=[8, 8],words:=["11", "22", "33",
"1313", "12323212132132", "1212121212121212", "2323232323232323",
"1321212321323212121"]),
# https://www.abstract-polytopes.com/atlas/128/1135/1.html
sha256=285dc01b8d5cbdcf365adb217fa994e9efb1459dd1b8a22b233a5026d4a6ede8
rec(label:="{4,4,4}*128",order:=128,typ:=[4, 4, 4],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "12121212", "12321232", "23232323",
"23432343", "34343434"]),
# https://www.abstract-polytopes.com/atlas/256/16888/1.html
sha256=6a3cedc26c36e029c1c80d03ffe6535b618db988a503279f27f0d2245cce91bc
rec(label:="{4,4,4}*256a",order:=256,typ:=[4, 4, 4],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "12121212", "12321232", "23232323",
"34343434", "423423423423"]),
# https://www.abstract-polytopes.com/atlas/256/16888/2.html
sha256=752c60d899773e711f4e65a4acf5e35c11ee8d0d083f36a9b4ac8564e3a31605
rec(label:="{4,4,4}*256b",order:=256,typ:=[4, 4, 4],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "12121212", "23232323", "23432343",
"34343434", "312312312312"]),
# https://www.abstract-polytopes.com/atlas/256/16836/1.html
sha256=54306bcecafd7878e378e462deb55e6c379f491ab2d015e2c61cd9308828c8f1
rec(label:="{4,4,8}*256a",order:=256,typ:=[4, 4, 8],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "12121212", "12321232", "23232323",
"23432343", "3434343434343434"]),
# https://www.abstract-polytopes.com/atlas/256/16864/1.html
sha256=493fab45cc43ae4d632e1b41f2fed4b226a5ed5d9af819ca969385b0ecde028a
rec(label:="{4,4,8}*256b",order:=256,typ:=[4, 4, 8],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "12121212", "12321232", "23232323",
"4234342343"]),
# https://www.abstract-polytopes.com/atlas/256/16885/1.html
sha256=75bdbc71fbb33b76a9ded0cbc81f95b2b4dddab587bda987ae7fa48651b79a90
rec(label:="{4,8,4}*256a",order:=256,typ:=[4, 8, 4],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "12121212", "34343434", "3123231232",
"2342323423"]),
# https://www.abstract-polytopes.com/atlas/256/16894/1.html
sha256=dc6945485b1af225af4698c12a5268e293ef9f1a702b44362c38a3c99f4886f5
```

```
rec(label:="{4,8,4}*256b",order:=256,typ:=[4, 8, 4],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "12121212", "12321232", "34343434",
"2342323423"]),
# https://www.abstract-polytopes.com/atlas/256/16894/2.html
sha256=2f0772c9b0aa496076cfa6b4509dcbc275923ef20729e26aef559442d9a08a36
rec(label:="{4,8,4}*256c",order:=256,typ:=[4, 8, 4],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "12121212", "23432343", "34343434",
"3123231232"]),
# https://www.abstract-polytopes.com/atlas/256/16906/1.html
sha256=6d7d45f6cd5eedb1e52a6fd3dd7003422fc627bf60769c8859dd98be3b53b7fc
rec(label:="{4,8,4}*256d",order:=256,typ:=[4, 8, 4],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "12121212", "12321232", "23432343",
"34343434", "2323232323232323"]),
# https://www.abstract-polytopes.com/atlas/256/16836/2.html
sha256=96aab1dd903b3b3e04822d9e156ddecdad6f6a8d6442f739fb92c1bb30a1b82d
rec(label:="{8,4,4}*256a",order:=256,typ:=[8, 4, 4],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "12321232", "23232323", "23432343",
"34343434", "1212121212121212"]),
# https://www.abstract-polytopes.com/atlas/256/16864/2.html
sha256=2b2dddf86816e09641d75f64c9dd7ea5b7415933b7876556c51e0eb92a428c27
rec(label:="{8,4,4}*256b",order:=256,typ:=[8, 4, 4],words:=["11", "22",
"33", "44", "1313", "1414", "2424", "23232323", "23432343", "34343434",
"1231212312"]),
# https://www.abstract-polytopes.com/atlas/512/60809/1.html
sha256=78d50057c869f47e122b5191495d50807507c38e092f4b9e3299f4eb6310e23e
rec(label:="{4,4}*512",order:=512,typ:=[4, 4],words:=["11", "22", "33",
"1313", "12121212", "23232323", "123212321232123212321232123212321232"]),
# https://www.abstract-polytopes.com/atlas/512/58326/1.html
sha256=30d77168174f8d730b82156ff65e01b5374c545f8181b7b6774f773f18cad1c2
rec(label:="{4,8}*512a",order:=512,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "1232123212321232", "2323232323232323",
"31232323123123232312"]),
# https://www.abstract-polytopes.com/atlas/512/58338/1.html
sha256=2862c62ea84ae95380f7f859d1fa10b8c752e5c5b192bc9c351e2c7e0685c891
rec(label:="{4,8}*512b",order:=512,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "1232323212323232", "2323232323232323",
"312312312312312312312312"]),
# https://www.abstract-polytopes.com/atlas/512/58338/2.html
sha256=fd7c33c3acc540ebbfa59631f0cab485affbe2428e59ce7da8e38e76e23773eb
rec(label:="{4,8}*512c",order:=512,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "2323232323232323", "1232312321231232",
"31232323123123232312"]),
# https://www.abstract-polytopes.com/atlas/512/60821/1.html
sha256=6783aec8b02b2b7e05f05e3e25443915e56b5eec7e7ac7d0c0354651fec4af65
rec(label:="{4,8}*512d",order:=512,typ:=[4, 8],words:=["11", "22", "33",
"1313", "12121212", "1232323212323232", "2323232323232323",
"23123212132312312312312321"])
];;
# Table6: oriented labels, raw universal order (0 denotes the retained
infinite row), class count.
# Exactly four printed right-hand {4,16} types are corrected to their
{16,4} duals.
rows78:=[
[["{4,4}*32", "{4,4}*256"], 1024, 1, [4, 4, 4]],
[["{4,4}*64", "{4,4}*64"], 1024, 1, [4, 4, 4]],
[["{4,4}*128", "{4,4}*64"], 0, 2, [4, 4, 4]],
[["{4,4}*32", "{4,8}*256a"], 1024, 1, [4, 4, 8]],
[["{4,4}*32", "{4,8}*256b"], 1024, 1, [4, 4, 8]],
```

```
[["{4,4}*32", "{4,8}*256c"], 1024, 1, [4, 4, 8]],
[["{4,4}*32", "{4,8}*256d"], 1024, 1, [4, 4, 8]],
[["{4,4}*64", "{4,8}*128a"], 2048, 2, [4, 4, 8]],
[["{4,4}*64", "{4,8}*128b"], 2048, 2, [4, 4, 8]],
[["{4,4}*128", "{4,8}*64a"], 1024, 1, [4, 4, 8]],
[["{4,4}*128", "{4,8}*64b"], 1024, 1, [4, 4, 8]],
[["{4,4}*32", "{4,16}*256a"], 1024, 1, [4, 4, 16]],
[["{4,4}*32", "{4,16}*256b"], 1024, 1, [4, 4, 16]],
[["{4,4}*64", "{4,16}*128a"], 1024, 1, [4, 4, 16]],
[["{4,4}*64", "{4,16}*128b"], 1024, 1, [4, 4, 16]],
[["{4,4}*32", "{4,32}*256a"], 1024, 1, [4, 4, 32]],
[["{4,4}*32", "{4,32}*256b"], 1024, 1, [4, 4, 32]],
[["{4,8}*64a", "{8,4}*256a"], 1024, 1, [4, 8, 4]],
[["{4,8}*64a", "{8,4}*256b"], 1024, 1, [4, 8, 4]],
[["{4,8}*64a", "{8,4}*256c"], 1024, 1, [4, 8, 4]],
[["{4,8}*64a", "{8,4}*256d"], 1024, 1, [4, 8, 4]],
[["{4,8}*64b", "{8,4}*256a"], 1024, 1, [4, 8, 4]],
[["{4,8}*64b", "{8,4}*256b"], 1024, 1, [4, 8, 4]],
[["{4,8}*64b", "{8,4}*256c"], 1024, 1, [4, 8, 4]],
[["{4,8}*64b", "{8,4}*256d"], 1024, 1, [4, 8, 4]],
[["{4,8}*128a", "{8,4}*128a"], 2048, 2, [4, 8, 4]],
[["{4,8}*128a", "{8,4}*128b"], 2048, 2, [4, 8, 4]],
[["{4,8}*128b", "{8,4}*128b"], 2048, 2, [4, 8, 4]],
[["{4,8}*64a", "{8,8}*256a"], 1024, 1, [4, 8, 8]],
[["{4,8}*64a", "{8,8}*256b"], 1024, 1, [4, 8, 8]],
[["{4,8}*64a", "{8,8}*256c"], 1024, 1, [4, 8, 8]],
[["{4,8}*64a", "{8,8}*256d"], 1024, 1, [4, 8, 8]],
[["{4,8}*64a", "{8,8}*256e"], 1024, 1, [4, 8, 8]],
[["{4,8}*64a", "{8,8}*256f"], 1024, 1, [4, 8, 8]],
[["{4,8}*64a", "{8,8}*256g"], 1024, 1, [4, 8, 8]],
[["{4,8}*64a", "{8,8}*256h"], 1024, 1, [4, 8, 8]],
[["{4,8}*64b", "{8,8}*256a"], 1024, 1, [4, 8, 8]],
[["{4,8}*64b", "{8,8}*256b"], 1024, 1, [4, 8, 8]],
[["{4,8}*64b", "{8,8}*256c"], 1024, 1, [4, 8, 8]],
[["{4,8}*64b", "{8,8}*256d"], 1024, 1, [4, 8, 8]],
[["{4,8}*64b", "{8,8}*256e"], 1024, 1, [4, 8, 8]],
[["{4,8}*64b", "{8,8}*256f"], 1024, 1, [4, 8, 8]],
[["{4,8}*64b", "{8,8}*256g"], 1024, 1, [4, 8, 8]],
[["{4,8}*64b", "{8,8}*256h"], 1024, 1, [4, 8, 8]],
[["{4,8}*128a", "{8,8}*128a"], 1024, 1, [4, 8, 8]],
[["{4,8}*128a", "{8,8}*128b"], 1024, 1, [4, 8, 8]],
[["{4,8}*128a", "{8,8}*128c"], 1024, 1, [4, 8, 8]],
[["{4,8}*128a", "{8,8}*128d"], 1024, 1, [4, 8, 8]],
[["{4,8}*128b", "{8,8}*128a"], 1024, 1, [4, 8, 8]],
[["{4,8}*128b", "{8,8}*128b"], 1024, 1, [4, 8, 8]],
[["{4,8}*128b", "{8,8}*128c"], 1024, 1, [4, 8, 8]],
[["{4,8}*128b", "{8,8}*128d"], 1024, 1, [4, 8, 8]],
[["{4,8}*64a", "{8,16}*256a"], 1024, 1, [4, 8, 16]],
[["{4,8}*64a", "{8,16}*256b"], 1024, 1, [4, 8, 16]],
[["{4,8}*64a", "{8,16}*256c"], 1024, 1, [4, 8, 16]],
[["{4,8}*64a", "{8,16}*256d"], 1024, 1, [4, 8, 16]],
[["{4,8}*64a", "{8,16}*256e"], 1024, 1, [4, 8, 16]],
[["{4,8}*64a", "{8,16}*256f"], 1024, 1, [4, 8, 16]],
[["{4,8}*64b", "{8,16}*256c"], 1024, 1, [4, 8, 16]],
[["{4,8}*64b", "{8,16}*256d"], 1024, 1, [4, 8, 16]],
[["{4,8}*64b", "{8,16}*256e"], 1024, 1, [4, 8, 16]],
[["{4,8}*64b", "{8,16}*256f"], 1024, 1, [4, 8, 16]],
[["{4,16}*128a", "{16,4}*256a"], 1024, 1, [4, 16, 4]],
```

```
[["{4,16}*128a", "{16,4}*256b"], 1024, 1, [4, 16, 4]],
[["{4,16}*128b", "{16,4}*256a"], 1024, 1, [4, 16, 4]],
[["{4,16}*128b", "{16,4}*256b"], 1024, 1, [4, 16, 4]],
[["{4,16}*128a", "{16,8}*256a"], 1024, 1, [4, 16, 8]],
[["{4,16}*128a", "{16,8}*256b"], 1024, 1, [4, 16, 8]],
[["{4,16}*128a", "{16,8}*256c"], 1024, 1, [4, 16, 8]],
[["{4,16}*128a", "{16,8}*256d"], 1024, 1, [4, 16, 8]],
[["{4,16}*128a", "{16,8}*256e"], 1024, 1, [4, 16, 8]],
[["{4,16}*128a", "{16,8}*256f"], 1024, 1, [4, 16, 8]],
[["{4,16}*128b", "{16,8}*256a"], 1024, 1, [4, 16, 8]],
[["{4,16}*128b", "{16,8}*256b"], 1024, 1, [4, 16, 8]],
[["{4,16}*128b", "{16,8}*256c"], 1024, 1, [4, 16, 8]],
[["{4,16}*128b", "{16,8}*256d"], 1024, 1, [4, 16, 8]],
[["{4,16}*128b", "{16,8}*256e"], 1024, 1, [4, 16, 8]],
[["{4,16}*128b", "{16,8}*256f"], 1024, 1, [4, 16, 8]],
[["{4,32}*256a", "{32,4}*256a"], 1024, 1, [4, 32, 4]],
[["{4,32}*256a", "{32,4}*256b"], 1024, 1, [4, 32, 4]],
[["{4,32}*256b", "{32,4}*256b"], 1024, 1, [4, 32, 4]],
[["{8,4}*64a", "{4,8}*128a"], 1024, 1, [8, 4, 8]],
[["{8,4}*64a", "{4,8}*128b"], 1024, 1, [8, 4, 8]],
[["{8,4}*64b", "{4,8}*128a"], 1024, 1, [8, 4, 8]],
[["{8,4}*64b", "{4,8}*128b"], 1024, 1, [8, 4, 8]],
[["{8,4}*64a", "{4,16}*128a"], 1024, 1, [8, 4, 16]],
[["{8,4}*64a", "{4,16}*128b"], 1024, 1, [8, 4, 16]],
[["{8,4}*64b", "{4,16}*128a"], 1024, 1, [8, 4, 16]],
[["{8,4}*64b", "{4,16}*128b"], 1024, 1, [8, 4, 16]],
[["{8,8}*128a", "{8,8}*128a"], 1024, 1, [8, 8, 8]],
[["{8,8}*128a", "{8,8}*128b"], 1024, 1, [8, 8, 8]],
[["{8,8}*128a", "{8,8}*128c"], 1024, 1, [8, 8, 8]],
[["{8,8}*128a", "{8,8}*128d"], 1024, 1, [8, 8, 8]],
[["{8,8}*128b", "{8,8}*128a"], 1024, 1, [8, 8, 8]],
[["{8,8}*128b", "{8,8}*128b"], 1024, 1, [8, 8, 8]],
[["{8,8}*128b", "{8,8}*128d"], 1024, 1, [8, 8, 8]],
[["{8,8}*128c", "{8,8}*128a"], 1024, 1, [8, 8, 8]],
[["{8,8}*128c", "{8,8}*128d"], 1024, 1, [8, 8, 8]],
[["{8,8}*128d", "{8,8}*128d"], 1024, 1, [8, 8, 8]],
[["{4,4,4}*128", "{4,4,4}*256a"], 1024, 1, [4, 4, 4, 4]],
[["{4,4,4}*256a", "{4,4,4}*256b"], 1024, 1, [4, 4, 4, 4]],
[["{4,4,4}*128", "{4,4,8}*256a"], 1024, 1, [4, 4, 4, 8]],
[["{4,4,4}*128", "{4,4,8}*256b"], 1024, 1, [4, 4, 4, 8]],
[["{4,4,8}*256a", "{4,8,4}*256b"], 1024, 1, [4, 4, 8, 4]],
[["{4,4,8}*256a", "{4,8,4}*256d"], 1024, 1, [4, 4, 8, 4]],
[["{4,4,8}*256b", "{4,8,4}*256a"], 1024, 1, [4, 4, 8, 4]],
[["{4,4,8}*256b", "{4,8,4}*256c"], 1024, 1, [4, 4, 8, 4]]
];;
```

phase78_n009_integration_finite.g

```
# Source-bound raw presentations, all N009 finite table labels and
universal orders.
# All orders below come from COMPLETE Todd--Coxeter coset tables, not NQ
bounds.
SizeScreen([240,40]);;
report24:=[];;meet24:=0;;tc24:=0;;
Cox24:=function(f,typ)
 local n,rels,i,j;n:=Length(typ)+1;rels:=List(f{[1..n]},x->x^2);
 for i in [1..n] do for j in [i+1..n] do
```

```
  if j=i+1 then Add(rels,(f[i]*f[j])^typ[i]);else
  Add(rels,Comm(f[i],f[j]));fi;
 od;od;return rels;
end;;
TableWords24:=function(f)
 local a,b,c;a:=f[1];b:=f[2];c:=f[3];
 return [[(a*b*c*b)^4,(c*a*(b*c)^3*a*b)^2],
 [(c*a*b)^8,(a*(b*c)^3*b)^2],
 [(c*(a*b*c*b)^2)^2,(c*a*(b*c)^3*a*b)^2],
 [(a*(b*c)^3*b)^2,(c*a*b)^5*c*(b*a)^2*(c*b)^2*a*b]];
end;;
Build24:=function(name,F,rels,rank)
 local fg,table,n,gs,G,mark,sets,pars,i,j,typ;
 fg:=GeneratorsOfGroup(F);table:=CosetTableFromGensAndRels(fg,rels,[]);n:
 =Length(table[1]);
 gs:=List([1..Length(fg)],i->PermList(table[2*i-1]));G:=Group(gs);mark:=g
 s{[1..rank]};
 if Size(G)<>n or Subgroup(G,mark)<>G then Error("Complete regular coset
 action/order/generation",name);fi;
 if not ForAll(rels,w->MappedWord(w,fg,gs)=One(G)) then Error("Raw
 relators",name);fi;
 sets:=Combinations([1..rank]);pars:=List(sets,S->Subgroup(G,mark{S}));
 for i in [1..Length(sets)] do for j in [1..Length(sets)] do
  if Intersection(pars[i],pars[j])<>pars[Position(sets,Intersection(sets[
  i],sets[j]))] then Error("Actual IP",name);fi;
  meet24:=meet24+1;
 od;od;
 typ:=List([1..rank-1],i->Order(mark[i]*mark[i+1]));
 Add(report24,[name,n,typ]);tc24:=tc24+1;
 Print("PRESENTATION ",name," order=",n," type=",typ," IP=PASS\n");
 return rec(name:=name,group:=G,mark:=mark,allgens:=gs,order:=n,F:=F,rels
 :=rels);
end;;
Same24:=function(A,B)
 local
 h;h:=GroupHomomorphismByImages(A.group,B.group,A.mark,B.mark);return
 h<>fail and IsBijective(h);
end;;

f:=[];;model:=fail;;pair78:=[];;
Read("phase78_n009_integration_data.g");;
for A in atlas78 do
 A.words:=List(A.words,w->List(w,c->Int([c])));
 F:=FreeGroup(Length(A.typ)+1);f:=GeneratorsOfGroup(F);
 rel:=List(A.words,w->Product(List(w,i->f[i])));
 model:=Build24(Concatenation("Atlas_",A.label),F,rel,Length(f));
 if model.order<>A.order or
 List([1..Length(A.typ)],i->Order(model.mark[i]*model.mark[i+1]))<>A.typ
 then Error("Complete Atlas order/type",A.label);fi;
 A.group:=model.group;A.gens:=model.mark;
od;
atlas22:=Filtered(atlas78,A->Length(A.typ)=2 and A.order<512);;
atlas4_22:=Filtered(atlas78,A->Length(A.typ)=3);;
atlas24:=Filtered(atlas78,A->Length(A.typ)=2 and A.order=512);;
if [Length(atlas22),Length(atlas4_22),Length(atlas24)]<>[56,11,5] then
Error("Library partition");fi;
Identify78:=function(H,gs,library)
 local typ,matches,A,hom;
```

```
 typ:=List([1..Length(gs)-1],i->Order(gs[i]*gs[i+1]));matches:=[];
 for A in library do
  if A.order=Size(H) and A.typ=typ then
   hom:=GroupHomomorphismByImages(A.group,H,A.gens,gs);
   if hom<>fail and IsBijective(hom) then Add(matches,A.label);fi;
  fi;
 od;
 if Length(matches)<>1 then Error("Unique ordered marked
 label",matches,typ,Size(H));fi;
 return matches[1];
end;;
duals78:=List(Concatenation(atlas22,atlas4_22),A->[A.label,Identify78(A.g
roup,Reversed(A.gens),Concatenation(atlas22,atlas4_22))]);;
Dual78:=function(label) return First(duals78,x->x[1]=label)[2];end;;
Canon78:=function(pair) return
Minimum(pair,[Dual78(pair[2]),Dual78(pair[1])]);end;;
F:=FreeGroup(3);f:=GeneratorsOfGroup(F);words:=TableWords24(f);;
base24:=List([1..4],i->Build24(Concatenation("Table2_512",[CharInt(96+i)]
),F,Concatenation(Cox24(f,[4,8]),words[i]),3));;
if not ForAll(base24,A->A.order=512) then Error("Table2 orders");fi;
for i in [1..4] do for j in [i+1..4] do if Same24(base24[i],base24[j])
then Error("Distinct Table2 markings");fi;od;od;
low24:=Build24("Example3.9_classI_base",F,Cox24(f,[2,8]),3);;
if low24.order<>32 then Error("ClassI boundary");fi;
q44:=Build24("Example3.9_classII_base",F,Concatenation(Cox24(f,[4,4]),[(f
[1]*f[2]*f[3]*f[2])^8]),3);;
if q44.order<>512 then Error("ClassII base");fi;
F4:=FreeGroup(4);f4:=GeneratorsOfGroup(F4);t:=f4[4];a:=f4[1];b:=f4[2];c:=
f4[3];;
central24:=Concatenation([t^2],List(f4{[1..3]},x->Comm(t,x)));;
classII24:=[];;
for eps in [0,1] do
 A:=Build24(Concatenation("Example3.9_classII_",String(eps)),F4,Concatena
 tion(Cox24(f4,[4,8]),central24,[t^-1*(b*c)^4,(a*b*c*b)^8*t^-eps]),3);;
 if A.order<>1024 or Order(A.allgens[4])<>2 then Error("ClassII
 lift");fi;Add(classII24,A);
od;
words4:=TableWords24(f4);patterns24:=[[1,0],[0,1],[1,1]];lifts24:=[];;pi:
=fail;;
for i in [1..4] do
 row:=[];
 for eps in patterns24 do
  A:=Build24(Concatenation("Table3_",String(i),"_",String(eps)),F4,Concat
  enation(Cox24(f4,[4,8]),central24,[words4[i][1]*t^-eps[1],words4[i][2]*
  t^-eps[2]]),3);;
  Add(row,A);
  N:=Subgroup(A.group,[A.allgens[4]]);pi:=NaturalHomomorphismByNormalSubg
  roup(A.group,N);Q:=Image(pi);;
  qm:=List(A.mark,x->Image(pi,x));h:=GroupHomomorphismByImages(base24[i].
  group,Q,base24[i].mark,qm);;
  if h=fail or not IsBijective(h) then Error("Lift quotient is exact
  Table2 marking");fi;
 od;Add(lifts24,row);
od;
# Names a,b from Example3.9 ClassII; c,d,e,f,g,h from the explicit Table3
assignments.
refs24:=[classII24[1],classII24[2],lifts24[1][1],lifts24[1][2],lifts24[1]
[3],lifts24[2][3],lifts24[3][2],lifts24[3][3]];;
```

```
if not ForAll(refs24,A->A.order=1024) then Error("Eight target
orders");fi;
for i in [1..8] do for j in [i+1..8] do if Same24(refs24[i],refs24[j])
then Error("Eight targets not distinct",i,j);fi;od;od;
expected24:=[[3,4,5],[1,3,6],[3,7,8],[9,1,9]];;table3matches24:=[];;
for i in [1..4] do for j in [1..3] do
 k:=expected24[i][j];if k=9 then B:=base24[4];else B:=refs24[k];fi;
 if not Same24(lifts24[i][j],B) then Error("Exact Table3
 assignment",i,j,k);fi;
 Add(table3matches24,[i,patterns24[j],k,lifts24[i][j].order,Order(lifts24
 [i][j].allgens[4])]);
od;od;
# Literal Table4, preserving each printed e (identity) even when Table3
disagrees.
literal24:=[];;
for i in [1..8] do
 if i<=2 then rel:=Concatenation(Cox24(f4,[4,8]),central24,[t^-1*(b*c)^4,
 (a*b*c*b)^8*t^-(i-1)]);
 elif i in [3,5] then rel:=Concatenation(Cox24(f4,[4,8]),central24,[words
 4[1][1]*t^-1,words4[1][2]]);
 elif i=4 then rel:=Concatenation(Cox24(f4,[4,8]),central24,[words4[1][1]
 ,words4[1][2]*t^-1]);
 elif i=6 then rel:=Concatenation(Cox24(f4,[4,8]),central24,[words4[2][1]
 *t^-1,words4[2][2]]);
 elif i=7 then rel:=Concatenation(Cox24(f4,[4,8]),central24,[words4[3][1]
 ,words4[3][2]*t^-1]);
 else rel:=Concatenation(Cox24(f4,[4,8]),central24,[words4[3][1]*t^-1,wor
 ds4[3][2]]);fi;
 A:=Build24(Concatenation("Table4_literal_",String(i)),F4,rel,3);;
 matches:=Filtered([1..8],j->Same24(A,refs24[j]));if Length(matches)<>1
 then Error("Literal Table4 identification");fi;
 Add(literal24,[i,matches[1]]);
od;
if literal24<>[[1,1],[2,2],[3,3],[4,4],[5,3],[6,1],[7,7],[8,3]] then
Error("Unexpected literal Table4 mapping",literal24);fi;
Print("TABLE3_MATCHES
",table3matches24,"\n");Print("TABLE4_LITERAL_MATCHES ",literal24,"\n");
# Examples4.3--4.5, using exactly their printed raw relators.
a:=f4[1];b:=f4[2];c:=f4[3];d:=f4[4];;
ex43:=Build24("Example4.3",F4,Concatenation(Cox24(f4,[4,4,4]),[(a*b*c*b)^
2,(b*c*d*c)^4]),4);;
ex44:=Build24("Example4.4",F4,Concatenation(Cox24(f4,[4,4,4]),[(a*b*c*b)^
2,(d*b*c)^8]),4);;
ex45rel:=Concatenation(Cox24(f4,[4,4,8]),[(c*a*b)^4,d*b*c*d*(c*b)^2*(d*c)
^2*b*c]);;
ex45:=Build24("Example4.5_universal",F4,ex45rel,4);;
if [ex43.order,ex44.order,ex45.order]<>[512,1024,2048] then
Error("Example raw orders");fi;
tauwords24:=[b*c*d*c*b*a*b*(d*c)^2*d*b*a,(b*c*d*c*b*a)^2,(c*d)^4];;
tauimages24:=List(tauwords24,w->MappedWord(w,f4,ex45.mark));;
zinv24:=Filtered(Elements(Centre(ex45.group)),z->Order(z)=2);;
if Set(tauimages24)<>Set(zinv24) or Length(zinv24)<>3 then
Error("Complete central involution list");fi;
ex45quot24:=[];;
for i in [1..3] do
 A:=Build24(Concatenation("Example4.5_quotient",String(i)),F4,Concatenati
 on(ex45rel,[tauwords24[i]]),4);;
 if A.order<>1024 then Error("Quotient raw order");fi;Add(ex45quot24,A);
```

```
od;
if List(ex45quot24,A->List([1..3],i->Order(A.mark[i]*A.mark[i+1])))<>[[4,
4,8],[4,4,8],[4,4,4]] then Error("Quotient smoothness");fi;
if Same24(ex45quot24[1],ex45quot24[2]) then Error("Distinct smooth
quotients");fi;
# Primary Atlas identities for the five source order-512 names.

for i in [1..5] do
 A:=atlas24[i];B:=rec(group:=Group(A.gens),mark:=A.gens);
 if i=1 then target:=q44;else target:=base24[i-1];fi;
 if not Same24(target,B) then Error("Primary Atlas base
 label",A.label);fi;
 if not ForAll(A.words,w->Product(List(w,j->A.gens[j]))=One(B.group))
 then Error("Primary Atlas defining word",A.label);fi;
od;
Print("PASS five exact primary Atlas order-512 marked labels\n");
# Exact facet identities for Examples4.3/4.4/4.5 and the two smooth
quotients.

exampleFacets24:=[];;
for entry in [[ex43,["{4,4}*32","{4,4}*128"]],[ex44,["{4,4}*32","{4,4}*25
6"]],[ex45,["{4,4}*64","{4,8}*128a"]],[ex45quot24[1],["{4,4}*64","{4,8}*1
28a"]],[ex45quot24[2],["{4,4}*64","{4,8}*128a"]]] do
 A:=entry[1];
 for j in [1,2] do
  ar:=First(atlas22,x->x.label=entry[2][j]);B:=rec(group:=Group(ar.gens),
  mark:=ar.gens);
  mark:=A.mark{[j..j+2]};target:=rec(group:=Subgroup(A.group,mark),mark:=
  mark);
  if not Same24(target,B) then Error("Example facet
  label",A.name,entry[2][j]);fi;
 od;
 Add(exampleFacets24,[A.name,entry[2]]);
od;
Print("PASS ten exact primary Atlas example facet labels\n");
# Match the eight printed/Table3 target names to the independent
unfiltered census.
if LoadPackage("nq")=fail then Error("NQ required for unfiltered census
adapter");fi;
Print("SOFTWARE GAP=",GAPInfo.Version,"
NQ=",PackageInfo("nq")[1].Version,"\n");
accepted24:=EvalString(StringFile("phase78_d3_k5.accepted.json"));;
F3:=FreeGroup(3);v:=GeneratorsOfGroup(F3);P:=F3/Concatenation(List(v,x->x
^2),[Comm(v[1],v[3])]);;
ep:=NqEpimorphismNilpotentQuotient(P,7);is:=IsomorphismPcGroup(Image(ep))
;U:=Image(is);;
rr:=List(GeneratorsOfGroup(P),x->Image(is,Image(ep,x)));pc24:=Pcgs(U);;
if not ForAll(RelativeOrders(pc24),x->x=2) then Error("Frozen binary pc
coordinates");fi;
Decode24:=function(n) local e,i;e:=[];for i in [1..Length(pc24)] do
Add(e,n mod 2);n:=QuoInt(n,2);od;if n<>0 then Error("Coordinate
overflow");fi;return PcElementByExponents(pc24,e);end;;
censusMatches24:=[];;
for entry in Filtered(accepted24,x->x[2]=[4,8]) do
 N:=Subgroup(U,List(entry[1],Decode24));
 if not IsNormal(U,N) or Index(U,N)<>1024 then Error("Frozen census
 reconstruction");fi;
```

```
 pi:=NaturalHomomorphismByNormalSubgroup(U,N);mark:=List(rr,x->Image(pi,x
 ));target:=rec(group:=Image(pi),mark:=mark);
 matches:=Filtered([1..8],j->Same24(refs24[j],target));
 if Length(matches)<>1 then Error("Exact census match",matches);fi;
 Add(censusMatches24,[entry[1],matches[1]]);
od;
if Set(List(censusMatches24,x->x[2]))<>[1..8] or
Length(censusMatches24)<>8 then Error("All eight census targets");fi;
Print("PASS all eight type-{4,8} classes matched to new unfiltered
census\n");
PrintTo("phase78_n009_integration_example_facets.json",exampleFacets24,"\
n");
PrintTo("phase78_n009_integration_census_matches.json",censusMatches24,"\
n");
PrintTo("phase78_n009_integration_examples_results.json",report24,"\n");
PrintTo("phase78_n009_integration_table3_matches.json",table3matches24,"\
n");
PrintTo("phase78_n009_integration_table4_literal_matches.json",literal24,
"\n");

# Reconstruct both higher-rank catalogues from the new complete kernel
census.
facetRows78:=[];;tableResults78:=[];;universal78:=[];;exceptionTargets78:
=[];;
for d78 in [4,5] do
 excess78:=11-2*d78;
 accepted78:=EvalString(StringFile(Concatenation("phase78_d",String(d78),
 "_k",String(excess78),".accepted.json")));
 F:=FreeGroup(d78);f:=GeneratorsOfGroup(F);rel:=List(f,x->x^2);
 for i in [1..d78] do for j in [i+1..d78] do if j-i>1 then
 Add(rel,Comm(f[i],f[j]));fi;od;od;
 P:=F/rel;ep:=NqEpimorphismNilpotentQuotient(P,excess78+2);is:=Isomorphis
 mPcGroup(Image(ep));U:=Image(is);
 rr:=List(GeneratorsOfGroup(P),x->Image(is,Image(ep,x)));pc24:=Pcgs(U);
 if not ForAll(RelativeOrders(pc24),x->x=2) then Error("Binary census
 coordinates");fi;
 if d78=4 then library78:=atlas22;else library78:=atlas4_22;fi;
 for entry in accepted78 do
  N:=Subgroup(U,List(entry[1],Decode24));
  if not IsNormal(U,N) or Index(U,N)<>1024 then Error("Census
  reconstruction");fi;
  pi:=NaturalHomomorphismByNormalSubgroup(U,N);H:=Image(pi);mark:=List(rr
  ,x->Image(pi,x));
  labels78:=[Identify78(Subgroup(H,mark{[1..d78-1]}),mark{[1..d78-1]},lib
  rary78),Identify78(Subgroup(H,mark{[2..d78]}),mark{[2..d78]},library78)
  ];
  Add(facetRows78,[entry[1],entry[2],labels78,entry[5]]);
  if labels78=["{4,4}*128","{4,4}*64"] then
  Add(exceptionTargets78,rec(group:=H,mark:=mark));fi;
 od;
od;
if Length(facetRows78)<>214 or Length(exceptionTargets78)<>2 then
Error("Facet/exceptions cardinality");fi;
if Length(Set(List(rows78,r->Canon78(r[1]))))<>107 then Error("Distinct
Table6 pairs");fi;
for row78 in rows78 do
 pair78:=Canon78(row78[1]);hits78:=Filtered(facetRows78,r->Canon78(r[3])=
 pair78);
```

```
 fixed78:=Number(hits78,r->r[4]);actual78:=(Length(hits78)+fixed78)/2;
 if actual78<>row78[3] then Error("Exact Table6
 count",row78,actual78);fi;
 if not ForAll(hits78,r->Minimum(r[2],Reversed(r[2]))=Minimum(row78[4],Re
 versed(row78[4]))) then Error("Table6 type");fi;
 Add(tableResults78,[row78[1],Length(hits78),fixed78,actual78]);
 if row78[2]<>0 then
  A:=First(atlas78,x->x.label=row78[1][1]);B:=First(atlas78,x->x.label=ro
  w78[1][2]);
  F:=FreeGroup(Length(A.typ)+2);f:=GeneratorsOfGroup(F);
  rel:=Concatenation(List(A.words,w->Product(List(w,i->f[i]))),List(B.wor
  ds,w->Product(List(w,i->f[i+1]))),[Comm(f[1],f[Length(f)])]);
  model:=Build24(Concatenation("Universal_",A.label,"_",B.label),F,rel,Le
  ngth(f));
  if model.order<>row78[2] then Error("Complete universal
  order",row78);fi;
  if [Identify78(Subgroup(model.group,model.mark{[1..Length(f)-1]}),model
  .mark{[1..Length(f)-1]},atlas78),Identify78(Subgroup(model.group,model.
  mark{[2..Length(f)]}),model.mark{[2..Length(f)]},atlas78)]<>row78[1]
  then Error("Universal exact facet identities");fi;
  Add(universal78,[row78[1],model.order]);
 fi;
od;
# Exact source Theorem5.5 raw presentations; match both independently
found markings.
F:=FreeGroup(4);f:=GeneratorsOfGroup(F);a:=f[1];b:=f[2];c:=f[3];d:=f[4];
common78:=Concatenation(Cox24(f,[4,4,4]),[(a*b*c*b)^4,(b*c*d)^4]);;
extras78:=[[(d*c*b*a*b*d*c)^2],[(b*d*c*d*b*a)^2*(c*b*a*b)^2,(a*b*c*b)^2*(
b*a*b*d*c*d)^2]];;
exceptionMatches78:=[];;
for i in [1,2] do
 model:=Build24(Concatenation("Theorem5.5_",String(i)),F,Concatenation(co
 mmon78,extras78[i]),4);
 if model.order<>1024 then Error("Exceptional raw order");fi;
 matches78:=Filtered([1,2],j->Same24(model,exceptionTargets78[j]));
 if Length(matches78)<>1 then Error("Exceptional marked isomorphism");fi;
 Add(exceptionMatches78,matches78[1]);
od;
if Set(exceptionMatches78)<>[1,2] then Error("Exceptional
completeness");fi;
if Length(universal78)<>106 or
Collected(List(universal78,x->x[2]))<>[[1024,101],[2048,5]] then
Error("All finite universals");fi;
# Table5 counts, from the full rank3 census, only after generation is
complete.
expected578:=[[[4,4],1],[[4,8],8],[[4,16],14],[[4,32],4],[[4,64],2],[[4,1
28],2],[[8,8],22],[[8,16],40],[[8,32],8],[[8,64],4],[[16,16],10],[[16,32]
,10]];;
for row78 in expected578 do
 hits78:=Filtered(accepted24,r->Minimum(r[2],Reversed(r[2]))=row78[1]);
 if (Length(hits78)+Number(hits78,r->r[5]))/2<>row78[2] then
 Error("Table5 count");fi;
od;
PrintTo("phase78_n009_integration_facet_matches.json",facetRows78,"\n");
PrintTo("phase78_n009_integration_duals.json",duals78,"\n");
PrintTo("phase78_n009_integration_table6_results.json",tableResults78,"\n
");
```

```
PrintTo("phase78_n009_integration_universal_orders.json",universal78,"\n"
);
PrintTo("phase78_n009_integration_all_presentations.json",report24,"\n");
Print("PASS complete_presentations=",tc24," actual_IP_meets=",meet24,"
atlas=72 raw_examples=34 exceptional=2 finite_universals=106
exact_census_facets=428 table6_rows=107 table5_rows=12\n");
QUIT;
```

phase78_n009_integration_run.py

```
#!/usr/bin/env python3
"""Fresh exhaustive kernel census and complete-presentation recognition.

Inputs are exactly three printed GAP/data programs; no saved census,
private
primary text, Atlas permutation array or SmallGroups catalogue is read.
The output directory must not exist. GAP/NQ remain trusted exact
algorithms.
"""
import argparse
import hashlib
import json
from pathlib import Path
import re
import shutil
import subprocess
import time

ROOT = Path(__file__).resolve().parent
INPUTS = ["phase78_n009_integration_census.g",
"phase78_n009_integration_finite.g", "phase78_n009_integration_data.g"]
CASES = [(3, 3, 61, 37, 3, 20, 3904, [7, 23, 61]), (3, 5, 401, 235, 15,
125, 25664, [7, 23, 61, 147, 401]), (4, 3, 1483, 199, 11, 105, 379648,
[31, 267, 1483]), (5, 1, 127, 15, 1, 8, 130048, [127])]
FINAL = "PASS complete_presentations=214 actual_IP_meets=43840 atlas=72
raw_examples=34 exceptional=2 finite_universals=106
exact_census_facets=428 table6_rows=107 table5_rows=12"


def require(condition, message):
    if not condition:
        raise RuntimeError(message)


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def main():
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--gap", default="gap")
    ap.add_argument("--output-dir", type=Path, required=True)
    ap.add_argument("--timeout", type=int, default=1200)
    args = ap.parse_args()
    out = args.output_dir.resolve()
    require(not out.exists(), "Refusing existing output directory")
    bindings = [{"path": n, "sha256": sha(ROOT / n)} for n in INPUTS]
```

```
out.mkdir(parents=True)
start = time.monotonic()
for n in INPUTS:
    shutil.copy2(ROOT / n, out / n)
timings = {}
for name, program in [("census", INPUTS[0]), ("finite", INPUTS[1])]:
    t = time.monotonic()
    run = subprocess.run([args.gap, "-q", "-T", program], cwd=out,
    text=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
    timeout=args.timeout, check=False)
    (out / (name + ".log")).write_text(run.stdout)
    (out / (name + ".err")).write_text(run.stderr)
    require(run.returncode == 0 and not run.stderr.strip(), f"{name}:
    execution error or warning")
    require(not re.search(r"(^|\n)(Error,|Syntax error:|brk>)",
    run.stdout), f"{name}: GAP failure")
    timings[name] = round(time.monotonic() - t, 6)
census = (out / "census.log").read_text()
finite = (out / "finite.log").read_text()
require(census.count("PASS: complete invariant-kernel ladder") == 4
and FINAL in finite, "Incomplete terminal markers")
observed = re.findall(r"RESULT rank=(\d+) excess=(\d+)
candidates=(\d+) ordered=(\d+) selfdual=(\d+) up_to_reversal=(\d+)
actual_meets=(\d+)", census)
require([tuple(map(int, r)) for r in observed] == [x[:7] for x in
CASES], "Census totals differ")
for d, k, candidates, ordered, fixed, classes, meets, levels in
CASES:
    for j, n in enumerate(levels, 1):
        data = json.loads((out /
        f"phase78_d{d}_k{k}.level{j}.json").read_text())
        require(len(data) == n and len({tuple(x) for x in data}) ==
        n, "Ladder cardinality or duplicate key")
    accepted = json.loads((out /
    f"phase78_d{d}_k{k}.accepted.json").read_text())
    require(len(accepted) == ordered and sum(x[4] for x in accepted)
    == fixed, "Accepted cardinality/fixed count")
seeds = json.loads((out / "phase78_d3_k3.seed.json").read_text())
require(len(seeds) == 4 and sorted(x[1] for x in seeds) == [1, 2, 3,
4], "All four distinct seed rows")
read = lambda suffix: json.loads((out / ("phase78_n009_integration_"
+ suffix + ".json")).read_text())
require(len(read("all_presentations")) == 214, "Complete presentation
count")
require(read("table4_literal_matches") == [[1, 1], [2, 2], [3, 3],
[4, 4], [5, 3], [6, 1], [7, 7], [8, 3]], "Literal correction map")
require(len(read("table3_matches")) == 12, "Table3 cells")
require(len(read("facet_matches")) == 214 and
len(read("table6_results")) == 107, "Facet/table6 rows")
require(len(read("universal_orders")) == 106, "Finite universals")
require(len(read("census_matches")) == 8 and sorted(x[1] for x in
read("census_matches")) == list(range(1, 9)), "Eight named source
targets")
versions = re.findall(r"VERSION GAP=(\S+) NQ=(\S+)", census)
require(len(versions) == 4 and len(set(versions)) == 1, "Software
identity")
for row in bindings:
```

```
        require(sha(ROOT / row["path"]) == row["sha256"] == sha(out /
        row["path"]), "Mathematical input changed during replay")
    files = [{"path": p.name, "sha256": sha(p)} for p in
    sorted(out.iterdir()) if p.is_file()]
    report = dict(status="PASS", software=dict(zip(["GAP", "NQ"],
    versions[0])), elapsed_seconds=round(time.monotonic()-start, 6),
    timings=timings,
        cases=[dict(zip(["rank", "excess", "terminal_kernels", "ordered",
        "selfdual", "classes", "actual_meets", "ladder_sizes"], c)) for c
        in CASES],
        total_census_meets=539264, complete_presentations=214,
        presentation_IP_meets=43840,
        source_table6_rows=107, source_table5_rows=12,
        named_census_facet_isomorphisms=428,
        source_statement_scope="Corrected principal contract with
        explicitly retained geometric/square/infinite inputs; no
        historical novelty or Lean verification.",
        candidate_generation_uses_source_totals=False,
        SmallGroups_dependency=False, old_census_input=False,
        private_primary_input=False,
        inputs=bindings, runner=dict(path=Path(__file__).name,
        sha256=sha(Path(__file__))), outputs=files)
    (out /
    "phase78_n009_integration_run.json").write_text(json.dumps(report,
    indent=2)+"\n")
    print(json.dumps({k: report[k] for k in ["status", "software",
    "elapsed_seconds", "total_census_meets", "complete_presentations",
    "presentation_IP_meets"]}))


if __name__ == "__main__":
    main()
```